BİL 468 / BİL 587 – PROJE SON RAPORU

Erişim Bilgileri
Orijinal Veri Seti Adı: polyvore-outfits
Orijinal Veri Seti Bağlantısı: https://huggingface.co/datasets/mvasil/polyvore-outfits
Proje Çalışmasının Yüklendiği Google Drive Bağlantısı: https://drive.google.com/drive/folders/11Rzcho1VD7kS21rd-v_MzKYwA664NRY6
GitHub/Repo Bağlantısı: https://github.com/Zehraatalay/stylematch/tree/main/FinalProject

Çalıştırılacak Dosyalar ve Açıklamaları:
1- config.py: Veri, split, output ve seed paths/settings tanımlar.
2- make_strict_disjoint.py: Train-valid-test arasında item overlap kalmayacak şekilde strict splits üretir.
3- data_prep.py : Outfitlerden dengeli pozitif/negatif kıyafet çiftleri üretir.
4- dataset.py : Parquet içindeki image'ları okuyup scratch/transfer preprocessing ve augmentation'u uygular.
5- model_scratch.py: Pretrained ağırlık kullanmayan early-fusion CNN tabanlı compatibility modelini tanımlar.
6- model_transfer.py: ImageNet pretrained ResNet-18 tabanlı Siamese transfer-learning modelini tanımlar.
7- train_scratch.py: Scratch modelinin tuning/final training, early stopping, ve test evaluation'unu yürütür.
8- train_transfer.py : Transfer modelini 2 aşamada (head + fine-tuning) eğitir ve değerlendirir.
9- train_utils.py : Training loop, ROC-AUC tabanlı checkpoint/early stopping ve metrics calculation içerir.
10- tune.py: Scratch ve Transfer modelleri için seçilmiş hyperparameters deneylerini çalıştırır.
11- inspect_dataset.py: Dataset ve split yapısını kontrol eder.
12- smoke_test.py: Scratch ve transfer pipeline'larının küçük veri üzerinde çalıştığını doğrular.
13- demo.py: Eğitilmiş modeller kullanılarak örnek compatibility tahmini gerçekleştirir.

Dosyaların Çalıştırılma Sırası :
1. make_strict_disjoint.py
2. data_prep.py
3. inspect_dataset.py
4. smoke_test.py
5. tune.py
6. train_scratch.py
7. train_transfer.py
8. demo.py

1. Proje Konusu

Bu projede Polyvore Outfits veri setindeki iki moda ürününün görsel olarak birbiriyle uyumlu olup olmadığının binary classification ile tahmin edilmesi amaçlanmıştır. Aynı problem için iki farklı CNN tabanlı yaklaşım karşılaştırılmıştır. Yöntem 1'de iki görüntünün kanal boyutunda birleştirildiği ve tamamen sıfırdan eğitilen bir early-fusion CNN modeli kullanılmıştır. Yöntem 2'de ise iki görüntünün ortak ağırlıklı ImageNet pretrained ResNet-18 encoder ile ayrı ayrı işlendiği Siamese transfer-learning mimarisi kullanılmıştır. Her iki model de sonuçta iki ürünün uyumluluğunu temsil eden tek bir logit üretmektedir.

2. Literatür Taraması

Moda ürünleri arasındaki uyumluluğun tahmini için literatürde Polyvore Outfit datasetini kullanan çeşitli çalışmalar yer almaktadır. Han vd. (2017), Learning Fashion Compatibility with Bidirectional LSTMs çalışmasında kıyafet kombinlerini sıralı bir yapı olarak ele almış ve Bidirectional LSTM (Bi-LSTM) ile ürünler arasındaki uyumluluğu öğrenmiştir. Ayrıca görsel ve anlamsal özelliklerin embedding temsillerinden yararlanmış ve yöntemin outfit compatibility ile fill-in-the-blank görevlerinde başarılı sonuçlar verdiğini göstermiştir.

Vasileva vd. (2018) ise Learning Type-Aware Embeddings for Fashion Compatibility çalışmasında ayakkabı-pantolon veya üst-çanta gibi farklı ürün türleri arasındaki uyumluluğun farklı özelliklere bağlı olabileceğini dikkate almıştır. Bu nedenle type-aware embedding yaklaşımını önermiş ve Polyvore üzerinde outfit compatibility ve fill-in-the-blank görevlerinde önceki yöntemlere göre daha başarılı sonuçlar elde etmiştir.

Bu çalışmalardan hareketle projemizde ürünlerin görsel özelliklerinden compatibility tahmini yapılmış; sıfırdan eğitilen early-fusion CNN ile pretrained ResNet-18 tabanlı Siamese transfer-learning yaklaşımı karşılaştırılmıştır.

3. Veri Seti ve Değerlendirme Metrikleri

Polyvore Outfits'in disjoint bölünmesi temel alınmıştır. Ek veri sızıntısını engellemek için test seti sabit tutulmuş; test ile item paylaşan validation outfitleri, ardından temizlenmiş validation veya test ile item paylaşan training outfitleri çıkarılmıştır. Son strict splitlerde train-valid, train-test ve valid-test item overlap değerleri 0'dır.

Pair oluşturma: Pozitif örnekler, aynı outfit içerisinde yer alan farklı kategorilerdeki iki üründen oluşturulmuştur. Negatif örnek oluşturulurken ilk ürün sabit tutulmuş, ikinci ürün ise aynı kategoriye ait ancak farklı bir outfit içerisinden seçilmiştir. Böylece modelin yalnızca ürün kategorisine bakarak pozitif/negatif ayrımı yapmasının önüne geçilmiştir. Her pozitif pair için bir negatif pair üretilerek veri dengeli tutulmuştur.

Split	Outfit	Unique Item	Üretilen Pairs	Pozitif	Negatif
Train	11379	51042	86608	43304	43304
Validation	2954	14457	22598	11299	11299
Test	15145	70035	111462	55731	55731

Tuning aşamasında hesaplama maliyetini sınırlamak için 20,000 training ve 5,000 validation çifti kullanılmıştır. Final eğitimde max_train_pairs ve max_valid_pairs kaldırılarak yukarıdaki tüm çiftler kullanılmış, test değerlendirmesi de 111,462 test çiftinin tamamında yapılmıştır.

Değerlendirme metrikleri: binary cross-entropy loss, accuracy, precision, recall, F1-score ve ROC-AUC. Model seçimi ve early stopping için temel metrik ROC-AUC olarak kullanılmıştır.

4. Yöntem 1

Yöntem 1'de herhangi bir pretrained model veya pretrained ağırlık kullanılmamış, CNN mimarisi tamamen sıfırdan eğitilmiştir. İki RGB ürün görüntüsü özellik çıkarımından önce kanal boyutunda birleştirilerek 6 kanallı tek bir giriş oluşturulmuştur (**early fusion**). Bu giriş, farklı kernel boyutlarıyla özellik çıkaran multi-scale başlangıç katmanından ve ardışık CNN bloklarından geçirilmiştir. Kanal sayısı `base_channels` parametresine bağlı olarak kademeli biçimde artırılmış, son convolutional özellikler Global Average Pooling ile tek bir özellik vektörüne dönüştürülmüştür. Elde edilen temsil fully-connected katmanlardan geçirilerek iki ürün için tek bir compatibility logiti üretilmiştir. Eğitim görüntülerinde augmentation uygulanırken validation ve test aşamalarında augmentation uygulanmamıştır.

- Final seçim: AdamW, LR=3e-4, weight decay=1e-4, dropout=0.4, embed_dim=128, base_channels=24, batch size=64.
- Final eğitim: En fazla 8 epoch, patience=3; tüm 86,608 training ve 22,598 validation çifti kullanılmıştır.
- En iyi validation checkpoint: Epoch 8, ROC-AUC=0.6844.

Scratch Early-Fusion CNN'de Değiştirilen Parametreler

Bu yöntemde CNN mimarisi sıfırdan eğitilmiş ve model geliştirme sürecinde aşağıdaki hiperparametreler karşılaştırılmıştır:

- **Optimizer:** Adam ve AdamW.
- **Learning rate:** 3e-4.
- **Dropout:** 0.3 ve 0.4.
- **Weight decay:** 0 ve 1e-4.

- **Embedding boyutu:** 128 ve 256.
- **CNN kapasitesi / base channel sayısı:** 24 ve 32.
- **Early stopping:** Tuning aşamasında validation ROC-AUC üzerinden patience=2 kullanılmıştır. Final eğitimde patience=3 kullanılmıştır.

Deneyler:

- **S1_baseline:** Adam, dropout=0.3, weight decay=0, embed_dim=128, base_channels=24.
- **S2_adamw:** AdamW, dropout=0.4, weight decay=1e-4, embed_dim=128, base_channels=24.
- **S3_capacity:** AdamW, dropout=0.3, weight decay=1e-4, embed_dim=256, base_channels=32.

Validation ROC-AUC açısından en iyi tuning sonucu **S2_adamw** ile elde edilmiş ve final Yöntem 1 modeli bu konfigürasyon kullanılarak eğitilmiştir.

5. **Yöntem 2**

Yöntem 2'de ortak encoder olarak ImageNet ağırlıklarıyla başlatılan ResNet-18 kullanılmış ve orijinal classification katmanı kaldırılmıştır. 512-boyutlu backbone çıktısı 256 ara boyut üzerinden embedding'e projeksiyonlanmıştır. İki görüntüden elde edilen embedding'ler için |e_a-e_b|, e_a*e_b ve e_a+e_b olmak üzere üç simetrik pair özelliği hesaplanmış ve birleştirilerek fully-connected karşılaştırma katmanlarına verilmiştir. Eğitim iki aşamalıdır: önce backbone tamamen dondurularak yalnızca projection/compare head eğitilmiş, ardından layer4 açılarak düşük learning rate ile fine-tuning yapılmıştır.

- Final seçim: AdamW, embed_dim=256, dropout=0.4, weight decay=1e-4, unfreeze=layer4.
- Head LR=3e-4; fine-tuning head LR=1e-4; backbone LR=1e-5; batch=64.
- Final eğitim: 2 head epoch + en fazla 6 fine-tuning epoch, patience=3; tüm train/validation çiftleri.
- Fine-tuning epoch 2'de en iyi validation ROC-AUC=0.6459; üç epoch iyileşme olmayınca early stopping.

ResNet-18 Siamese Transfer Learning'de Değiştirilen Parametreler:

Bu yöntemde ResNet-18 omurgası sabit bırakılmamış; farklı optimizer, embedding ve fine-tuning stratejileri denenmiştir:

- **Optimizer:** Adam ve AdamW karşılaştırıldı.
- **Embedding boyutu:** 128 ve 256 denendi.
- **Dropout:** `0.3` ve `0.4` değerleri denendi.
- **Backbone learning rate:** `1e-5` ve `3e-6` değerleri denendi.
- **Fine-tuning derinliği:** yalnızca `layer4` açma ile `layer3 + layer4` birlikte açma karşılaştırıldı.
- **Weight decay:** `1e-4` kullanıldı.

- **İki aşamalı eğitim:** önce backbone dondurulduu ve yalnızca head eğitildi; sonra seçilen ResNet katmanları açılarak fine-tuning yapıldı.
- **Early stopping:** validation ROC-AUC üzerinden `patience=2` kullanıldı.

Deneyler:

- **T1_baseline:** Adam, embed_dim=128, dropout=0.3, layer4 fine-tuning, backbone LR=`1e-5`
- **T2_adamw:** AdamW, embed_dim=256, dropout=0.4, layer4 fine-tuning, backbone LR=`1e-5`
- **T3_deeper:** AdamW, embed_dim=128, dropout=0.3, layer3+layer4 fine-tuning, backbone LR=`3e-6`

En iyi tuning sonucu validation ROC-AUC açısından T2_adamw ile elde edilmiştir.

6. Deney Sonuçları ve Değerlendirme

6.1. Hiperparametre tuning sonuçları

Deney	Optimizer	LR/Backbone LR	Dropout	Embed	Kapasite/Unfreeze	Val Acc.	Val F1	Val AUC
S1_Baseline	Adam	3e-4	0.3	128	base_ch=24	0.6034*	0.5675*	0.6361
S2_AdamW	AdamW	3e-4	0.4	128	base_ch=24	0.6040	0.5984	0.6420
S3_Capacity	AdamW	3e-4	0.3	256	base_ch=32	0.5938	0.5287	0.6295
T1_Baseline	Adam	1e-5	0.3	128	Layer 4	0.5842	0.6020	0.6176
T2_AdamW	AdamW	1e-5	0.4	256	Layer 4	0.5800	0.6070	0.6177
T3_Deeper	AdamW	3e-6	0.3	128	Layer 3+4	0.5816	0.6012	0.6170

* S1 için tabloda en yüksek ROC-AUC'nin elde edildiği epoch 3'ün accuracy/F1 değerleri verilmiştir. Diğer satırlarda da seçim ölçütü ROC-AUC'dir. Tuning 20k/5k çift üzerinde ve test setine bakılmadan yapılmıştır.

Scratch deneylerinde en iyi ROC-AUC S2_adamw (0.6420) ile elde edildiği için final scratch modeli bu ayarlarla seçilmiştir. Transfer deneylerinde T1, T2 ve T3 birbirine çok yakın sonuç vermiş; en yüksek ROC-AUC 0.6177 ile T2_adamw olduğundan final transfer

modeli olarak T2 seçilmiştir. Daha derin unfreeze (layer3+layer4) ek kazanç sağlamamıştır.

6.2. Final eğitim ve test sonuçları

Model	En İyi Val AUC	Test Loss	Test Acc.	Precision	Recall	F1	Test AUC
Scratch S2_final	0.6844	0.6471	0.6202	0.6372	0.5583	0.5951	0.6651
Transfer T2_final	0.6459	0.6646	0.5965	0.5913	0.6246	0.6075	0.6346

Final sonuçlarda scratch model ROC-AUC ve accuracy açısından daha başarılıdır (AUC 0.6651'e karşı 0.6346; accuracy 0.6202'ye karşı 0.5965). Transfer modelinin F1'i biraz daha yüksektir (0.6075'e karşı 0.5951); bunun temel nedeni recall değerinin daha yüksek olmasıdır (0.6246). Scratch model ise daha yüksek precision (0.6372) üretmiştir. Dolayısıyla genel ayırt etme gücü açısından ROC-AUC ölçütüne göre Yöntem 1 daha başarılıdır.

Scratch model 8. epocha kadar validation ROC-AUC'yi geliştirmeye devam etmiştir. Transfer modelde ise fine-tuning epoch 2 sonrasında validation ROC-AUC 0.6459 -> 0.6457 -> 0.6441 -> 0.6406 biçiminde düşmüş ve patience=3 ile eğitim erken durdurulmuştur. Bu davranış, transfer modelinde sonraki epochların genelleme performansını artırmadığını göstermektedir.

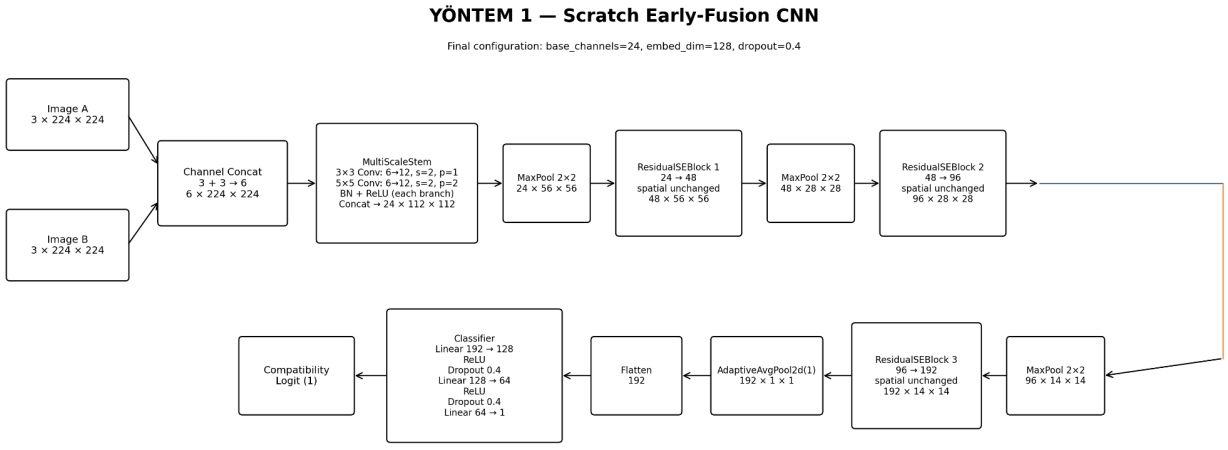
7. Sonuç ve Potansiyel Geliştirmeler

Proje kapsamında sıfırdan eğitilen early-fusion CNN ve pretrained ResNet-18 tabanlı Siamese transfer-learning modeli aynı strict-disjoint veri düzeninde karşılaştırılmıştır. Beklentinin aksine pretrained ResNet-18 tabanlı transfer model, ROC-AUC bakımından scratch CNN'i geçememiştir. Nihai scratch model test ROC-AUC=0.6651 ile en iyi genel sonucu vermiştir. Transfer model ise daha yüksek recall ve F1-score elde etmiştir. Her iki modelin performansı, yalnızca iki ürünün görsel özelliklerinden moda uyumluluğunu tahmin etmenin zor bir problem olduğunu göstermektedir. Gelecekte kategori bilgisinin modele eklenmesi, hard-negative mining, learning-rate scheduler, daha kapsamlı hiperparametre optimizasyonu ve farklı backbone veya pair-comparison tasarımları denenebilir. Ayrıca karar threshold'u validation seti üzerinde optimize edilerek precision-recall dengesi iyileştirilebilir.

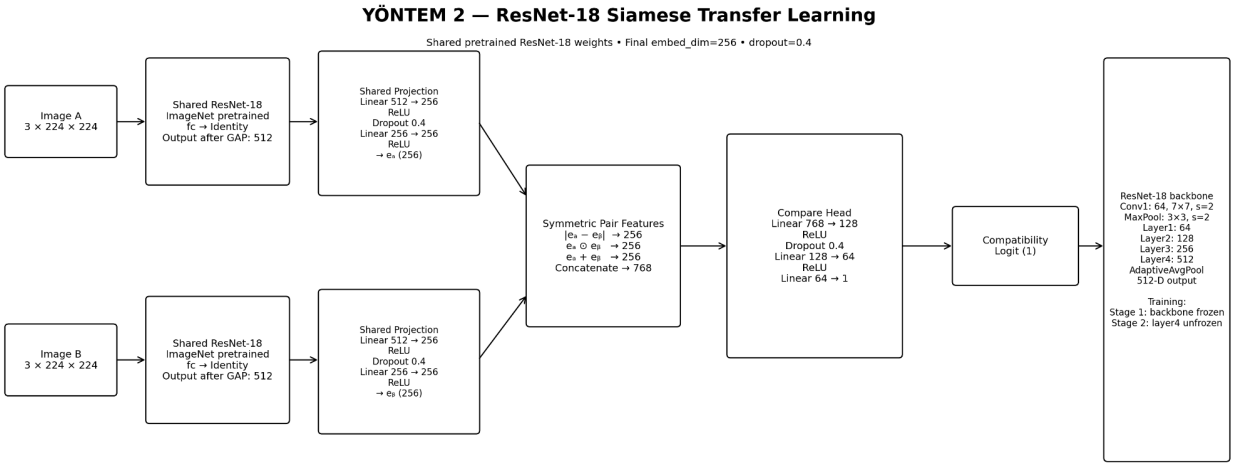
8. Kaynakça

[1] Han, X. et al., “Learning Fashion Compatibility with Bidirectional LSTMs,” ACM Multimedia, 2017.
[2] Vasileva, M. I. et al., “Learning Type-Aware Embeddings for Fashion Compatibility,” ECCV, 2018.
[3] Bromley, J. et al., “Signature Verification using a Siamese Time Delay Neural Network,” NeurIPS, 1993.
[4] He, K. et al., “Deep Residual Learning for Image Recognition,” CVPR, 2016.
[5] PyTorch/Torchvision documentation, ResNet-18 and transfer-learning utilities.

9. Ekler



Şekil 1. Yöntem 1 kapsamında kullanılan sıfırdan eğitilen early-fusion CNN tabanlı compatibility modelinin mimarisi.



Şekil 2. Yöntem 2 kapsamında kullanılan pretrained ResNet-18 tabanlı Siamese compatibility modelinin mimarisi